

0. 基础板子

1. 数据结构

- 1.1 并查集
 - 1.1.1 带权并查集
- 1.2 树状数组
- 1.3 segtree
- 1.4 Lazysegtree
- 1.5 动态开点线段树
- 1.6 可持久化线段树
- 1.7 ST表
- 1.8 笛卡尔树
- 1.9 FHQTreap
- 1.10 普通莫队

2. 图论

- 2.1 kruskal求最小生成树
 - 2.1.1 重构树
- 2.2 Prim求最小生成树
- 2.3 dijkstra
- 2.4 Floyd
- 2.5 匈牙利算法
- 2.6 Tarjan求有向图强连通分量
- 2.7 缩点
- 2.8 dinic
- 2.9 欧拉路径
 - 2.9.1 无向图欧拉路径
 - 2.9.2 有向图欧拉路径

3. 树

- 3.1 lca(倍增)
- 3.2 lca(tarjan)
- 3.3 重链剖分
- 3.4 树上启发式合并
- 3.5 树哈希

4. 数学

- 4.1 快速幂和组合数
- 4.2 线性筛
- 4.3 exgcd
- 4.4 线性求逆
- 4.5 线性基
- 4.6 矩阵类
- 4.7 FFT
- 4.8 卡特兰数
- 4.9 MillerRabin素性检验
- 4.10 CRT

5. 字符串

- 5.1 Hash
- 5.2 KMP
- 5.3 Manacher
- 6.4 Trie
- 6.5 ACAutomation
- 6.6 SA

6. DP

- 6.1 子集枚举
- 6.2 斜率优化

7. 博弈论

- 7.1 Nim博弈
- 7.2 Wythoff博弈

8. 杂项

- 8.1 最长上升子序列
- 8.2 三分
 - 8.2.1 三分 (三等分)
 - 8.2.2 三分 (近似三分)
 - 8.2.3 三分 (整数三分)
- 8.3 CDQ分治
- 8.4 离散化
- 8.5 随机数
- 8.6 III的输出
- 8.7 对拍
- 8.8 设置输出精度
- 8.9 pbds的平衡树

0. 基础板子

```
1  #include <bits/stdc++.h>
2  #include <bits/extc++.h>
3  using namespace std;
4  // using namespace __gnu_pbds;
5  using ll = long long;
6  using ull = unsigned long long;
7  using lll = __int128;
8  using pll = pair<ll, ll>;
9  #define fp(i, l, r) for(int i=(l); i<=(r); ++i)
10 #define fq(i, r, l) for(int i=(r); i>=(l); --i)
11 #define fi first
12 #define se second
13 #define debug(x) cerr << #x << ": " << x << '\n';
14 const int N = 2e5+10;
15 const ll MOD = 998244353;
16 const ll INF = 0x3f3f3f3f3f3f3f3f;
17 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
18
19 void solve(){
20
21 }
22
23 int main(){
24     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
25     int _ = 1;
26     // cin>> _;
27     while(--)
28         solve();
29     return 0;
30 }
```

1. 数据结构

1.1 并查集

```
1 struct DSU//启发式合并 + 路径压缩
2 {
3     int n;
4     vector<int> fa, sz;
5     explicit DSU(int n) : n(n)
6     {
7         fa.assign(n + 1, 0);
8         sz.assign(n + 1, 1);
9         for (int i = 1; i <= n; i++)
10             fa[i] = i;
11     }
12     void reset()
13     {
14         for(int i = 0; i <= n; ++i)
15             fa[i] = i, sz[i] = 1;
16     }
17     int find(int x)
18     {
19         if(fa[x] == x)
20             return x;
21         return fa[x] = find(fa[x]);
22     }
23     void merge(int x, int y)
24     {
25         int dx = find(x);
26         int dy = find(y);
27         if(dx != dy)
28         {
29             if(sz[dx] > sz[dy])
30                 swap(dx, dy);
31             sz[dy] += sz[dx];
32             fa[dx] = dy;
33         }
34     }
35     bool judge(int x, int y)
36     {
37         return find(x) == find(y);
38     }
39     int size(int x)
40     {
41         return sz[find(x)];
42     }
43 };
```

1.1.1 带权并查集

```
1 struct WDSU
2 {
3     int n;
```

```

4     vector<int> fa, sz;
5     vector<ll> d;
6     explicit WDSU(int n) : n(n)
7     {
8         fa.assign(n + 1, 0);
9         sz.assign(n + 1, 1);
10        d.assign(n + 1, 0);
11        for (int i = 1; i <= n; i++)
12            fa[i] = i;
13    }
14    void reset()
15    {
16        for(int i = 0; i <= n; ++i)
17            fa[i] = i, sz[i] = 1, d[i] = 0;
18    }
19    int find(int x)
20    {
21        if(fa[x] == x){
22            return x;
23        }
24        else{
25            int oldfa=fa[x];
26            fa[x] = find(fa[x]);
27            d[x] += d[oldfa];
28            return fa[x];
29        }
30    }
31    void merge(int x, int y, ll c)
32    {
33        int dx = find(x);
34        int dy = find(y);
35        if(dx != dy)
36        {
37            d[dx]=d[y]+c-d[x];
38            fa[dx]=dy;
39            sz[dy]+=sz[dx];
40        }
41    }
42    bool judge(int x, int y)
43    {
44        return find(x) == find(y);
45    }
46    int size(int x)
47    {
48        return sz[find(x)];
49    }
50 };

```

1.2 树状数组

```

1     template <class T>
2     struct Fenwick {
3         int n;

```

```

4     vector<T> a;
5     Fenwick(int n = 0): n(n), a(n + 1) {}
6     void add(int x, const T &v) {
7         for (int i = x; i <= n; i += i & -i) a[i] = a[i] + v;
8     }
9     T sum(int x) {
10        T ans=0;
11        for (int i = x; i > 0; i -= i & -i) ans = ans + a[i];
12        return ans;
13    }
14    T rangeSum(int l, int r) {
15        return sum(r) - sum(l - 1);
16    }
17    int get_kth(int k) {
18        int pos = 0;
19        for (int i = __lg(n); i >= 0; --i) {
20            int nxt = pos + (1 << i);
21            if (nxt <= n && a[nxt] < k) {
22                pos = nxt;
23                k -= a[pos];
24            }
25        }
26        return pos + 1;
27    }
28 };

```

1.3 segtree

```

1     struct SegTree {
2         struct Info {
3             ll val;
4             Info operator+(const Info& x) const { return {max(val, x.val)}; }
5             void operator+=(const Info& x) { *this = *this + x; }
6         };
7         int min_, max_;
8         vector<Info> info;
9         SegTree(int n): min_(1), max_(n), info(4 * n) {}
10        SegTree(int l, int r): min_(l), max_(r), info(4 * (r - l + 1)) {}
11    private:
12        void pull(int i) { info[i] = info[2 * i] + info[2 * i + 1]; }
13        void set(int pos, Info v, int i, int l, int r) {
14            if (l == r) {info[i] = v; return;}
15            if (pos <= l + (r - l) / 2) set(pos, v, 2 * i, l, l + (r - l) / 2);
16            else set(pos, v, 2 * i + 1, l + (r - l) / 2 + 1, r);
17            pull(i);
18        }
19        void add(int pos, Info v, int i, int l, int r) {
20            if (l == r) {info[i] += v; return;}
21            if (pos <= l + (r - l) / 2) add(pos, v, 2 * i, l, l + (r - l) / 2);
22            else add(pos, v, 2 * i + 1, l + (r - l) / 2 + 1, r);
23            pull(i);
24        }
25        Info query(int x, int y, int i, int l, int r) {
26            if (l >= x && r <= y) return info[i];

```

```

27         if (y <= 1 + (r - 1) / 2) return query(x, y, 2 * i, 1, 1 + (r - 1) /
28         2);
29         if (x > 1 + (r - 1) / 2) return query(x, y, 2 * i + 1, 1 + (r - 1) /
30         2 + 1, r);
31         return query(x, y, 2 * i, 1, 1 + (r - 1) / 2) + query(x, y, 2 * i +
32         1, 1 + (r - 1) / 2 + 1, r);
33     }
34     int binarySearch(int i, int l, int r) {
35         if (l == r) return l;
36         if (info[2 * i].val <= 0) return binarySearch(2 * i, 1, 1 + (r - 1)
37         / 2);
38         return binarySearch(2 * i + 1, 1 + (r - 1) / 2 + 1, r);
39     }
40     public:
41     void set(int i, const Info& v) { set(i, v, 1, min_, max_); }
42     void add(int i, const Info& v) { add(i, v, 1, min_, max_); }
43     Info query(int l, int r) { return query(l, r, 1, min_, max_); }
44     int binarySearch() { return binarySearch(1, min_, max_); }
45 };

```

1.4 Lazysegtree

```

1  struct LazySegTree {
2      struct Info {
3          ll val = 0;
4          Info operator+(const Info& x) const { return {min(val, x.val)}; }
5          void operator+=(const Info& x) { *this = *this + x; }
6      };
7      struct Lazy {
8          ll val = 0;
9          Lazy operator+(const Lazy& x) { return { val += x.val }; }
10         void operator+=(const Lazy& x) { *this = *this + x; }
11     };
12     int min_, max_;
13     vector<Info> info;
14     vector<Lazy> tag;
15     LazySegTree(int n): min_(1), max_(n), info(4 * n), tag(4 * n) {}
16     LazySegTree(int l, int r): min_(l), max_(r), info(4 * (r - l + 1)),
17     tag(4 * (r - l + 1)) {}
18     private:
19     void operation(const Lazy& lazy, int i, int l, int r) {
20         info[i].val += lazy.val;
21         tag[i] += lazy;
22     }
23     void pull(int i) { info[i] = info[2 * i] + info[2 * i + 1]; }
24     void push(int i, int l, int r) {
25         operation(tag[i], 2 * i, l, (l + r) / 2);
26         operation(tag[i], 2 * i + 1, (l + r) / 2 + 1, r);
27         tag[i] = {};
28     }
29     void set(int pos, const Info& v, int i, int l, int r) {
30         if (l == r) { info[i] = v; return; }
31         push(i, l, r);
32         if (pos <= (l + r) / 2) set(pos, v, 2 * i, l, (l + r) / 2);
33         else set(pos, v, 2 * i + 1, (l + r) / 2 + 1, r);

```

```

33     pull(i);
34 }
35 void rangeOperation(int x, int y, const Lazy& v, int i, int l, int r) {
36     if (y < x || l > y || r < x) return;
37     if (l >= x && r <= y) return operation(v, i, l, r);
38     push(i, l, r);
39     rangeOperation(x, y, v, 2 * i, l, (l + r) / 2);
40     rangeOperation(x, y, v, 2 * i + 1, (l + r) / 2 + 1, r);
41     pull(i);
42 }
43 Info query(int x, int y, int i, int l, int r) {
44     if (l >= x && r <= y) return info[i];
45     push(i, l, r);
46     if (y <= (l + r) / 2) return query(x, y, 2 * i, l, (l + r) / 2);
47     if (x > (l + r) / 2) return query(x, y, 2 * i + 1, (l + r) / 2 + 1,
r);
48     return query(x, y, 2 * i, l, (l + r) / 2) + query(x, y, 2 * i + 1,
(l + r) / 2 + 1, r);
49 }
50 public:
51     void set(int i, const Info& v) { set(i, v, 1, min_, max_); }
52     void rangeOperation(int l, int r, const Lazy& v) { rangeOperation(l, r,
v, 1, min_, max_); }
53     Info query(int l, int r) { return query(l, r, 1, min_, max_); }
54     friend ostream& operator<<(ostream& os, LazySegTree& seg) {
55         os << "seg[";
56         for (int i = seg.min_; i <= seg.max_; i++) {
57             os << seg.query(i, i).val << ", ";
58         }
59         os << "]";
60         return os;
61     }
62 };

```

1.5 动态开点线段树

```

1 struct SegmentTree {
2     struct Info {
3         ll sum = 0;
4         ll cnt = 0;
5         ll val = 0;
6         Info() {}
7         Info(ll v): sum(v), cnt(1) {}
8         Info(ll sum, ll cnt, ll val): sum(sum), cnt(cnt), val(val) {}
9         Info operator+(const Info& x) {
10             return {sum + x.sum, cnt + x.cnt, val + x.val + x.sum * cnt -
x.cnt * sum};
11         }
12         void operator+=(const Info& x) {
13             *this = *this + x;
14         }
15     };
16     struct Node {
17         Info info;
18         int ls = 0;

```

```

19     int rs = 0;
20     Node(){}
21     Node(const Info& x): info(x) {}
22 };
23 ll min_, max_;
24 vector<Node> node;
25
26 SegmentTree(ll l, ll r): min_(l), max_(r), node(2) {}
27 SegmentTree(): min_(0), max_(1e18), node(2) {}
28
29 private: //internal function part
30 void pull(int i) {
31     node[i].info = node[node[i].ls].info + node[node[i].rs].info;
32 }
33 void add(const ll& pos, const Info& val, int i, ll l, ll r) {
34     if (l == r) { node[i].info += val; return; }
35     ll m = l + (r - l) / 2;
36     if (pos <= m) {
37         if (node[i].ls == 0) {
38             node[i].ls = node.size();
39             node.emplace_back();
40         }
41         add(pos, val, node[i].ls, l, l + (r - l) / 2);
42     } else {
43         if (node[i].rs == 0) {
44             node[i].rs = node.size();
45             node.emplace_back();
46         }
47         add(pos, val, node[i].rs, l + (r - l) / 2 + 1, r);
48     }
49     pull(i);
50 }
51
52 Info query(const ll& x, const ll& y, int i, ll l, ll r) {
53     ll m = l + (r - l) / 2;
54     if (l >= x && r <= y) {
55         return node[i].info;
56     }
57     if (y <= m) {
58         return query(x, y, node[i].ls, l, l + (r - l) / 2);
59     } else if (x > m) {
60         return query(x, y, node[i].rs, l + (r - l) / 2 + 1, r);
61     } else {
62         return query(x, y, node[i].ls, l, l + (r - l) / 2) + query(x,
63 y, node[i].rs, l + (r - l) / 2 + 1, r);
64     }
65 }
66
67 ll kBiggest(int k, int i, ll l, ll r) {
68     if (l == r) {
69         return l;
70     }
71     if (node[i].rs == 0) {
72         return kBiggest(k, node[i].ls, l, l + (r - l) / 2);
73     }
74     int d = node[node[i].rs].info.sum;

```

```

74         if (k <= d) {
75             return kBiggest(k, node[i].rs, 1 + (r - 1) / 2 + 1, r);
76         } else {
77             return kBiggest(k - d, node[i].ls, 1, 1 + (r - 1) / 2);
78         }
79     }
80
81 public: // open interface part
82     void print(ostream& os, int i, ll l, ll r) const {
83         if (l == r) {
84             os << l << ": " << node[i].info.sum << endl;
85             return;
86         }
87         if (node[i].ls) {
88             print(os, node[i].ls, 1, 1 + (r - 1) / 2);
89         }
90         if (node[i].rs) {
91             print(os, node[i].rs, 1 + (r - 1) / 2 + 1, r);
92         }
93     }
94     void add(const ll& num) {
95         add(num, 1, 1, min_, max_);
96     }
97     Info query(const ll& x, const ll& y) {
98         return query(x, y, 1, min_, max_);
99     }
100     Info query() {
101         return query(min_, max_, 1, min_, max_);
102     }
103     ll kBiggest(int k) {return kBiggest(k, 1, min_, max_);}
104     friend ostream& operator<<(ostream& os, const SegmentTree& seg) {
105         seg.print(os, 0, seg.min_, seg.max_);
106         return os;
107     }
108 };

```

1.6 可持久化线段树

```

1  struct PST {
2      struct Info {
3          ll val = 0;
4          Info operator+(const Info& x) {
5              return {val + x.val};
6          }
7          void operator+=(const Info& x) {
8              *this = *this + x;
9          }
10     };
11     struct Node {
12         Info info;
13         int ls = 0;
14         int rs = 0;
15         Node operator+(const Info& x) {
16             Node nd = *this;
17             nd.info += x;

```

```

18         return nd;
19     }
20     Node() {}
21     Node(const Info& x) {
22         info=x;
23     }
24 };
25 int n = 0, min_ = 0, max_ = 1e9;
26 vector<Node> node;
27
28 PST(): node(1) {}
29 PST(int n): min_(1), max_(n), node(1) {
30     node.reserve(n*25);
31 }
32 PST(int l, int r): min_(l), max_(r), node(1) {}
33
34 private: //internal function part
35 void pull(int i) {
36     node[i].info = node[node[i].ls].info + node[node[i].rs].info;
37 }
38 int set(const int& pos, const Info& val, int old_version, int l, int r)
39 {
40     int cur = node.size();
41     Node tmp = node[old_version];
42     node.push_back(tmp);
43     if (l == r) {
44         node[cur].info = val;
45         return cur;
46     }
47     int m = l + (r - l) / 2;
48     if (pos <= m) {
49         int ls = set(pos, val, node[old_version].ls, l, m);
50         node[cur].ls = ls;
51     } else {
52         int rs = set(pos, val, node[old_version].rs, m + 1, r);
53         node[cur].rs = rs;
54     }
55     pull(cur);
56     return cur;
57 }
58 int add(const int& pos, const Info& val, int old_version, int l, int r)
59 {
60     int cur = node.size();
61     Node tmp = node[old_version];
62     node.push_back(tmp);
63     if (l == r) {
64         node[cur].info += val;
65         return cur;
66     }
67     int m = l + (r - l) / 2;
68     if (pos <= m) {
69         int ls = add(pos, val, node[old_version].ls, l, m);
70         node[cur].ls = ls;
71     } else {
72         int rs = add(pos, val, node[old_version].rs, m + 1, r);
73         node[cur].rs = rs;

```

```

72     }
73     pull(cur);
74     return cur;
75 }
76 Info query(const int& x, const int& y, int v, int l, int r) {
77     int m = l + (r - l) / 2;
78     if (l >= x && r <= y) {
79         return node[v].info;
80         debug(node[v].info.val);
81     }
82     if(y <= m){
83         return query(x, y, node[v].ls, l, m);
84     } else if(x > m){
85         return query(x, y, node[v].rs, m + 1, r);
86     } else {
87         return query(x, y, node[v].ls, l, m) + query(x, y, node[v].rs,
88 m + 1, r);
89     }
90 }
91 int build(const vector<ll>& a, int l, int r){
92     int cur = node.size();
93     node.push_back(Node());
94     if(l == r){
95         node[cur].info = {a[l]};
96         return cur;
97     }
98     int m = l + (r - l) / 2;
99     int ls = build(a, l, m);
100    int rs = build(a, m + 1, r);
101    node[cur].ls = ls;
102    node[cur].rs = rs;
103    pull(cur);
104    return cur;
105 }
106 void printVersion(int i, int l, int r) {
107     if (l == r) {
108         cout << '\t' << l << ": " << node[i].info.val << endl;
109         return;
110     }
111     int m = l + (r - l) / 2;
112     if (node[i].ls) {
113         printVersion(node[i].ls, l, m);
114     }
115     if (node[i].rs) {
116         printVersion(node[i].rs, m + 1, r);
117     }
118 }
119 public: // open interface part
120 int build(const vector<ll>& a){
121     return build(a, 1, a.size()-1);
122 }
123 int set(int i, const Info& v, int old_version) { // 返回值为对应版本号
124     return set(i, v, old_version, min_, max_);
125 }
126 int add(int i, const Info& v, int old_version) { // 返回值为对应版本号

```

```

127         return add(i, v, old_version, min_, max_);
128     }
129     Info query(int x, int y, int version) {
130         return query(x, y, version, min_, max_);
131     }
132     void printVersion(int i) {
133         cout << "print version " << i << ": " << endl;
134         printVersion(i, min_, max_);
135         cout << endl;
136     }
137 };
138

```

1.7 ST表

```

1  struct ST{
2      int n;
3      vector<vector<ll>> info;
4
5      ST(const vector<ll>& vec){
6          int n = vec.size() - 1;
7          int maxlog = __lg(n) + 1;
8          info.assign(maxlog + 1, vector<ll>(n + 1, 0));
9          for(int i=1;i<=n;++i){
10             info[0][i] = vec[i];
11         }
12         for(int j=1;j<=maxlog;++j){
13             for(int i=1;i+(1LL<<j)-1<=n;++i){
14                 info[j][i]=max(info[j-1][i], info[j-1][i+(1LL<<(j-1))]);
15             }
16         }
17     }
18
19     ll query(int l, int r){
20         int k = __lg(r-l+1);
21         return max(info[k][l], info[k][r-(1LL<<k)+1]);
22     }
23 };

```

1.8 笛卡尔树

```

1  void solve(){
2      ll n;
3      cin>>n;
4      vector<ll> p(n+1);
5      fp(i,1,n) cin>>p[i];
6      vector<ll> l(n+1);
7      vector<ll> r(n+1);
8      ll root=0;
9      stack<ll> stk;
10
11      fp(i,1,n){
12          ll pre=0;
13          while(stk.size() and p[i]<p[stk.top()]){

```

```

14         pre=stk.top();
15         stk.pop();
16     }
17     if(pre and stk.empty()){
18         l[i]=pre;
19         root=i;
20     }
21     else if(pre and stk.size()){
22         r[stk.top()]=i;
23         l[i]=pre;
24     }
25     else if(stk.size()){
26         r[stk.top()]=i;
27     }
28     else root=i;
29     stk.push(i);
30 }
31
32 ll ans1=0;
33 ll ans2=0;
34 fp(i,1,n){
35     ans1+=i*(l[i]+1);
36     ans2+=i*(r[i]+1);
37 }
38 cout<<ans1<<" "<<ans2;
39 }

```

1.9 FHQTreap

```

1  struct FHQTreap{
2      struct Node{
3          int l,r;
4          ll val,key;
5          int size;
6          Node(ll v,ll k=rng(),int sz=1){
7              val=v;
8              key=k;
9              size=sz;
10             l=r=0;
11         }
12     };
13     int root=0;
14     vector<Node> tr;
15
16     FHQTreap(){
17         tr.push_back(Node(0,0,0));
18     }
19
20     int newnode(ll val){
21         tr.push_back(Node(val));
22         return tr.size()-1;
23     }
24     void pushup(ll p){
25         tr[p].size=tr[tr[p].l].size+tr[tr[p].r].size+1;
26     }

```

```

27 void split(int p,ll val,int &x,int &y){
28     if(p==0) {
29         x=y=0;
30         return;
31     }
32     if(tr[p].val<=val){
33         x=p;
34         split(tr[p].r,val,tr[p].r,y);
35     }
36     else{
37         y=p;
38         split(tr[p].l,val,x,tr[p].l);
39     }
40     pushup(p);
41 }
42 int merge(int x,int y){
43     if(!x or !y) return x+y;
44     if(tr[x].key>tr[y].key){
45         tr[x].r=merge(tr[x].r,y);
46         pushup(x);
47         return x;
48     }
49     else{
50         tr[y].l=merge(x,tr[y].l);
51         pushup(y);
52         return y;
53     }
54 }
55 void insert(ll v){
56     int x,y;
57     split(root,v,x,y);
58     root=merge(merge(x,newnode(v)),y);
59 }
60 void del(ll v){
61     int x,y,z;
62     split(root,v,x,z);
63     split(x,v-1,x,y);
64     if(y) y=merge(tr[y].l,tr[y].r);
65     root=merge(merge(x,y),z);
66 }
67 int getrank(ll v){
68     int x,y;
69     split(root,v-1,x,y);
70     int res=tr[x].size+1;
71     root=merge(x,y);
72     return res;
73 }
74 ll getvalue(int rk){
75     int p=root;
76     while(p){
77         if(tr[tr[p].l].size+1==rk) return tr[p].val;
78         if(tr[tr[p].l].size>=rk) p=tr[p].l;
79         else{
80             rk-=tr[tr[p].l].size+1;
81             p=tr[p].r;
82         }

```

```

83     }
84     return -1;
85 }
86 ll getpre(ll v){
87     int x,y;
88     split(root,v-1,x,y);
89     int p=x;
90     while(tr[p].r) p=tr[p].r;
91     ll res=tr[p].val;
92     root=merge(x,y);
93     return res;
94 }
95 ll getnxt(ll v){
96     int x,y;
97     split(root,v,x,y);
98     int p=y;
99     while(tr[p].l) p=tr[p].l;
100    ll res=tr[p].val;
101    root=merge(x,y);
102    return res;
103 }
104 };

```

1.10 普通莫队

```

1  struct qry{
2      ll l,r,block,i;
3  };
4
5  bool cmp(qry a,qry b){
6      if(a.block!=b.block){
7          return a.block<b.block;
8      }
9      else if(a.block%2==0) return a.r<b.r;
10     else return a.r>b.r;
11 }
12
13 void solve(){
14     ll n,q;
15     cin>>n>>q;
16     vector<ll> p(n);
17     fp(i,0,n-1) cin>>p[i];
18     vector<qry> qrys;
19     ll d=(ll)sqrt(n);
20     fp(i,1,q){
21         ll l,r;
22         cin>>l>>r;
23         qrys.push_back({l-1,r-1,(l-1)/d,i});
24     }
25     sort(qrys.begin(),qrys.end(),cmp);
26
27     vector<ll> ans(q+1);
28     vector<ll> cnt(n+1);
29
30     ll curl=0;

```

```

31     ll curr=0;
32     ll curn=1;
33     cnt[p[0]]++;
34
35     auto add=[&](ll x){
36         if(cnt[p[x]]==0) curn++;
37         cnt[p[x]]++;
38     };
39
40     auto remove=[&](ll x){
41         if(cnt[p[x]]==1) curn--;
42         cnt[p[x]]--;
43     };
44
45     fp(i,0,q-1){
46         auto tar=qrys[i];
47         ll l=tar.l;
48         ll r=tar.r;
49         ll id=tar.i;
50         while(curl>l){
51             add(curl-1);
52             curl--;
53         }
54         while(curl<l){
55             remove(curl);
56             curl++;
57         }
58         while(curr<r){
59             add(curr+1);
60             curr++;
61         }
62         while(curr>r){
63             remove(curr);
64             curr--;
65         }
66         ans[id]=curn;
67     }
68
69     fp(i,1,q){
70         cout<<ans[i]<<"\n";
71     }
72 }

```

2. 图论

2.1 kruskal求最小生成树

```

1  struct edge{
2      ll x,y,z;
3  };
4
5  bool cmp(edge x,edge y){

```

```

6     return x.z<y.z;
7 }
8
9 void solve(){
10     ll n,m;
11     cin>>n>>m;
12     DSU dsu(n);
13     vector<edge> edges(m+1);
14     for(int i=1;i<=m;++i){
15         ll x,y,z;
16         cin>>x>>y>>z;
17         edges[i].x=x;
18         edges[i].y=y;
19         edges[i].z=z;
20     }
21     sort(edges.begin()+1,edges.end(),cmp);
22     ll cnt=0;
23     ll ans=0;
24     for(int i=1;i<=m;++i){
25         if(cnt==n-1) break;
26         ll x=edges[i].x;
27         ll y=edges[i].y;
28         ll z=edges[i].z;
29         if(!dsu.judge(x,y)){
30             cnt++;
31             ans+=z;
32             dsu.merge(x,y);
33         }
34     }
35     if(cnt<n-1) {cout<<"orz";return;}
36     cout<<ans;
37 }

```

2.1.1 重构树

```

1 struct ex_kruskal
2 {
3     int n,tot;
4     vector<int> fa; //DSU的fa数组
5     vector<ll> val;
6     vector<int> lc,rc;
7     vector<array<int, 22>> f; //重构树的fa倍增数组
8     vector<int> dep;
9     explicit ex_kruskal(int n) : n(n),tot(n)
10    {
11        fa.assign(2*n + 1, 0);
12        val.assign(2*n + 1, 0);
13        lc.assign(2*n + 1, 0);
14        rc.assign(2*n + 1, 0);
15        f.assign(2*n + 1,array<int, 22>{});
16        dep.assign(2*n + 1, 0);
17        for (int i = 1; i <= 2*n-1; i++)
18            fa[i] = i;
19    }
20    int find(int x)

```

```

21     {
22         if(fa[x] == x)
23             return x;
24         return fa[x] = find(fa[x]);
25     }
26 void merge(int x, int y, int w)
27 {
28     int dx = find(x);
29     int dy = find(y);
30     if(dx != dy)
31     {
32         fa[dx] = fa[dy] = ++tot;
33         val[tot]=w;
34         lc[tot]=dx;
35         rc[tot]=dy;
36     }
37 }
38 bool judge(int x, int y)
39 {
40     return find(x) == find(y);
41 }
42 void init(){
43     auto dfs=[&](auto&&self, ll u, ll d) ->void {
44         dep[u]=d;
45         if(lc[u]){
46             f[lc[u]][0]=u;
47             self(self, lc[u], d+1);
48         }
49         if(rc[u]){
50             f[rc[u]][0]=u;
51             self(self, rc[u], d+1);
52         }
53     };
54     for(int i=1; i<=tot; ++i){
55         if(fa[i]==i){
56             f[i][0]=i;
57             dfs(dfs, i, 1);
58         }
59     }
60     for(int j=1; j<=21; ++j) for(int i=1; i<=2*n-1; ++i)
61         f[i][j]=f[f[i][j-1]][j-1];
62 }
63 int lca(int x, int y)
64 {
65     if(dep[x]<dep[y]) swap(x,y);
66     while(dep[x] > dep[y])
67         x = f[x][__lg(dep[x] - dep[y])];
68     if(x==y) return x;
69     for(int i=21; i>=0; --i){
70         if(f[x][i]!=f[y][i]){
71             x=f[x][i];
72             y=f[y][i];
73         }
74     }
75     return f[x][0];
76 }

```

```

77
78     ll query(int x, int y){
79         if (!judge(x,y)) return -1;
80         return val[lca(x,y)];
81     }
82 };
83
84 void solve(){
85     ll n,q;
86     cin>>n>>q;
87     vector<ll> a(n+1);
88     fp(i,1,n) cin>>a[i];
89     vector<vector<ll>> has(N+1);
90     fp(i,1,n) has[a[i]].push_back(i);
91     ex_kruskal tr(n);
92     fq(i,N,1){
93         for(int j=0;j<=(int)has[i].size()-2;++j){
94             tr.merge(has[i][j],has[i][j+1],i);
95         }
96         ll pre=0;
97         for(int j=i;j<=N;j+=i){
98             if(has[j].size()){
99                 if(!pre) pre=has[j][0];
100                 tr.merge(pre,has[j][0],i);
101             }
102         }
103     }
104     tr.init();
105     while(q--){
106         ll x,y;
107         cin>>x>>y;
108         if(x==y) cout<<a[x]<<"\n";
109         else cout<<tr.query(x,y)<<"\n";
110     }
111 }

```

2.2 Prim求最小生成树

```

1 void solve(){
2     ll n,m;
3     cin>>n>>m;
4     vector<vector<p11>> G(n+1);
5     while(m--){
6         ll x,y,z;
7         cin>>x>>y>>z;
8         G[x].push_back({y,z});
9         G[y].push_back({x,z});
10    }
11    vector<bool> vis(n+1);
12    ll cnt=1;
13    ll ans=0;
14    vis[1]=1;
15    priority_queue<p11,vector<p11>,greater<p11>> pq;

```

```

16     for(auto [v,c]:G[1]){
17         pq.push({c,v});
18     }
19     while(!pq.empty()){
20         pll t=pq.top();
21         pq.pop();
22         ll c=t.fi;
23         ll v=t.se;
24         if(vis[v]) continue;
25         vis[v]=1;
26         ans+=c;
27         cnt++;
28         if(cnt==n) break;
29         for(auto [v1,c1]:G[v]){
30             if(vis[v1]) continue;
31             pq.push({c1,v1});
32         }
33     }
34     if(cnt==n) cout<<ans;
35     else cout<<"orz";
36 }

```

2.3 dijkstra

```

1  vector<ll> dijkstra(vector<vector<pll>> &G,ll n,ll st){
2      vector<ll> dis(n+1,INF);
3      vector<bool> vis(n+1,0);
4      priority_queue<pll,vector<pll>,greater<pll>> pq;
5      dis[st]=0;
6      pq.push({0LL,st});
7      while(!pq.empty()){
8          ll u=pq.top().second;
9          pq.pop();
10         if(vis[u]) continue;
11         vis[u]=1;
12         for(auto [v,w]:G[u]){
13             if(dis[u]+w<dis[v]){
14                 dis[v]=dis[u]+w;
15                 pq.push({dis[v],v});
16             }
17         }
18     }
19     return dis;
20 }
21
22 void solve(){
23     ll n,m,s;
24     cin>>n>>m>>s;
25     vector<vector<pll>> G(n+1);
26     for(int i=1;i<=m;++i){
27         ll u,v,w;
28         cin>>u>>v>>w;
29         G[u].push_back({v,w});
30     }
31     vector<ll> dis=dijkstra(G,n,s);

```

```

32     for(int i=1;i<=n;++i) {
33         cout<<dis[i]<<" ";
34     }
35
36 }
37
38 int main(){
39     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
40     int _ =1;
41     // cin>> _;
42     while(_--){
43         solve();
44     }
45     return 0;
46 }

```

2.4 Floyd

```

1 void solve(){
2     ll n,m;
3     cin>>n>>m;
4     vector<vector<ll>> a(n+1,vector<ll>(n+1,INF));
5     for(int i=1;i<=m;++i){
6         ll u,v,w;
7         cin>>u>>v>>w;
8         a[u][v]=min(a[u][v],w);
9         a[v][u]=min(a[v][u],w);
10    }
11    for(int i=1;i<=n;++i) a[i][i]=0;
12    for(int k=1;k<=n;++k){
13        for(int i=1;i<=n;++i){
14            for(int j=1;j<=n;++j){
15                a[i][j]=min(a[i][j],a[i][k]+a[k][j]);
16            }
17        }
18    }
19    for(int i=1;i<=n;++i){
20        for(int j=1;j<=n;++j){
21            cout<<a[i][j]<<" ";
22        }
23        cout<<"\n";
24    }
25 }

```

2.5 匈牙利算法

```

1 void solve(){
2     ll n,m,e;
3     cin>>n>>m>>e;
4     vector<vector<ll>> G(n+1);
5     fp(i,1,e){
6         ll u,v;
7         cin>>u>>v;

```

```

8      G[u].push_back(v);
9  }
10 vector<bool> vis(m+1);
11 vector<ll> match(m+1);
12 auto find=[&](auto&&self,ll x) ->bool {
13     for(auto y:G[x]){
14         if(vis[y]) continue;
15         vis[y]=1;
16         if(match[y]==0 or self(self,match[y])){
17             match[y]=x;
18             return 1;
19         }
20     }
21     return 0;
22 };
23 ll ans=0;
24 fp(i,1,n) {
25     vis.assign(m+1,0);
26     if(find(find,i)) ans++;
27 }
28 cout<<ans;
29 }

```

2.6 Tarjan求有向图强连通分量

```

1  vector<vector<ll>> g(N);
2  stack<ll> stk;
3  bitset<N> instk;
4  vector<ll> dfn(N),low(N);
5  ll timer_,scc_cnt;
6  vector<ll> scc_id(N),scc_sz(N);
7
8  void tarjan(ll u){
9      stk.push(u);
10     low[u]=dfn[u]=++timer_;
11     instk[u]=1;
12     for(auto &v:g[u]){
13         if(dfn[v]==0){
14             tarjan(v);
15             low[u]=min(low[u],low[v]);
16         }
17         else if(instk[v]){
18             low[u]=min(low[u],dfn[v]);
19         }
20     }
21
22     if(low[u]==dfn[u]){
23         ++scc_cnt;
24         while(1){
25             ll x=stk.top();
26             stk.pop();
27             scc_id[x]=scc_cnt;
28             instk[x]=0;
29             scc_sz[scc_cnt]++;
30             if(x==u) break;

```

```

31     }
32 }
33 }
34
35 void solve(){
36     ll n,m;
37     cin>>n>>m;
38     for(int i=1;i<=m;++i){
39         ll u,v;
40         cin>>u>>v;
41         g[u].push_back(v);
42     }
43     for(int i=1;i<=n;++i){
44         if(!dfn[i]) tarjan(i);
45     }
46     cout<<scc_cnt<<"\n";
47     vector<ll> call(n+1);
48     vector<vector<ll>> comp(scc_cnt);
49     for(int i=1;i<=n;++i){
50         if(call[i]) continue;
51         ll id=scc_id[i];
52         for(int j=1;j<=n;++j){
53             if(scc_id[j]==id){
54                 cout<<j<<" ";
55                 call[j]=1;
56             }
57         }
58         cout<<"\n";
59     }
60 }

```

2.7 缩点

```

1  vector<vector<ll>> g(N);
2  vector<ll> w(N);
3
4  stack<ll> stk;
5  bitset<N> instk;
6  vector<ll> dfn(N),low(N),scc_id(N);
7  ll timer_,scc_cnt;
8
9  vector<vector<ll>> dag(N);
10 vector<ll> indeg(N);
11 vector<ll> outdeg(N);
12 vector<ll> scc_w(N);
13
14 vector<ll> dp(N,-1);
15
16
17 void tarjan(ll u){
18     stk.push(u);
19     low[u]=dfn[u]=++timer_;
20     instk[u]=1;
21     for(auto &v:g[u]){
22         if(dfn[v]==0){

```

```

23         tarjan(v);
24         low[u]=min(low[u],low[v]);
25     }
26     else if(instk[v]){
27         low[u]=min(low[u],dfn[v]);
28     }
29 }
30
31 if(low[u]==dfn[u]){
32     ++scc_cnt;
33     while(1){
34         ll x=stk.top();
35         stk.pop();
36         scc_id[x]=scc_cnt;
37         instk[x]=0;
38         scc_w[scc_cnt]+=w[x];
39         if(x==u) break;
40     }
41 }
42 }
43
44 void solve(){
45     ll n,m;
46     cin>>n>>m;
47     for(int i=1;i<=n;++i){
48         cin>>w[i];
49     }
50     for(int i=1;i<=m;++i){
51         ll u,v;
52         cin>>u>>v;
53         g[u].push_back(v);
54     }
55
56     for(int i=1;i<=n;++i){
57         if(!dfn[i]) tarjan(i);
58     }
59
60     for(int i=1;i<=n;++i){
61         for(auto &x:g[i]){
62             ll u=scc_id[i];
63             ll v=scc_id[x];
64             if(u==v) continue;
65             dag[u].push_back(v);
66             indeg[v]++;
67             outdeg[u]++;
68         }
69     }
70     ll ans=0;
71     queue<ll> q;
72     for(int i=1;i<=scc_cnt;++i){
73         dp[i]=scc_w[i];
74         if(indeg[i]==0) q.push(i);
75     }
76     while(!q.empty()){
77         ll u=q.front();
78         q.pop();

```

```

79     ans=max(ans,dp[u]);
80     for(auto &v:dag[u]){
81         if(dp[v]<dp[u]+scc_w[v]){
82             dp[v]=dp[u]+scc_w[v];
83         }
84         indeg[v]--;
85         if(indeg[v]==0) q.push(v);
86     }
87 }
88 cout<<ans;
89 }
90
91 int main(){
92     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
93     int _=1;
94     // cin>> _;
95     while(_--){
96         solve();
97     }
98     return 0;
99 }

```

2.8 dinic

```

1  struct Edge{
2      int to, next;
3      ll w;
4  }edges[M];
5
6  int head[N];
7  int cnt=-1;
8
9  void add_edge(int u, int v, ll w){
10     edges[++cnt].to = v;
11     edges[cnt].w = w;
12     edges[cnt].next = head[u];
13     head[u] = cnt;
14 }
15
16 int n,m,s,t;
17
18 int lv[N],cur[N];
19
20 bool bfs(){
21     memset(lv, -1, sizeof(lv));
22     memcpy(cur, head, sizeof(head));
23     lv[s] = 0;
24     queue<int> q;
25     q.push(s);
26     while(q.size()){
27         int p=q.front();
28         q.pop();
29         for(int eg=head[p]; eg != -1; eg=edges[eg].next){
30             int to = edges[eg].to;
31             ll vol = edges[eg].w;
32             if(vol > 0 and lv[to] == -1){

```

```

33         lv[to] = lv[p] + 1;
34         q.push(to);
35     }
36 }
37 }
38 return (lv[t] != -1);
39 }
40
41 ll dfs(int p = s, ll flow = INF){
42     if(p == t) return flow;
43     ll rmn = flow;
44     for(int eg=cur[p]; (eg!=-1) and rmn; eg = edges[eg].next){
45         cur[p] = eg;
46         int to = edges[eg].to;
47         ll vol = edges[eg].w;
48         if(vol and lv[to] == lv[p] + 1){
49             ll c = dfs(to, min(vol, rmn));
50             rmn -= c;
51             edges[eg].w -= c;
52             edges[eg ^ 1].w += c;
53         }
54     }
55     return flow - rmn;
56 }
57
58 ll dinic(){
59     ll ans=0;
60     while(bfs()){
61         ans += dfs();
62     }
63     return ans;
64 }
65
66 void solve(){
67     cin>>n>>m>>s>>t;
68     memset(head, -1, sizeof(head));
69     fp(i,1,m){
70         ll u,v,w;
71         cin>>u>>v>>w;
72         add_edge(u,v,w);
73         add_edge(v,u,0);
74     }
75     cout<<dinic();
76 }

```

2.9 欧拉路径

欧拉路存在的充要条件如下：

首先，图是连通的。

对于无向图：

(1) 欧拉路径存在条件：有且仅有两个点，与其相连的边数为奇数，其他点相连边数皆为偶数；或所有点皆为偶数边点。

对于两个奇数点，一个为起点，一个为终点。

(2) 欧拉回路存在条件：如果存在这样一个欧拉路，其所有的点相连边数都为偶数，那说明它是欧拉回路。

对于有向图：

(1) 欧拉路径存在条件：除去起点和终点，所有点的出度与入度相等。起点出度比入度大1，终点入度比出度大1。

(2) 欧拉回路存在条件：若起点终点出入度也相同，则为欧拉回路。

2.9.1 无向图欧拉路径

```
1 void solve(){
2     ll n,m;
3     cin>>n>>m;
4     vector<vector<pll>> G(n+1);
5     vector<ll> cur(n+1);
6     vector<ll> deg(n+1);
7     vector<bool> vis(m+1);
8     ll tot=-1;
9     DSU dsu(n);
10    fp(i,1,m){
11        ll u,v;
12        cin>>u>>v;
13        G[u].push_back({v,++tot});
14        G[v].push_back({u,tot});
15        dsu.merge(u,v);
16        deg[u]++;
17        deg[v]++;
18    }
19    ll cnt=0;
20    ll st=0;
21    fp(i,1,n){
22        if(deg[i]%2!=0){
23            cnt++;
24            if(st==0) st=i;
25        }
26    }
27    if(cnt!=0 and cnt!=2){
28        cout<<"No\n";
29        return;
30    }
31    if(st==0){
32        fp(i,1,n) if(deg[i]!=0) {st=i;break;}
33    }
34    ll root=-1;
35    fp(i,1,n){
36        if(deg[i]==0) continue;
37        ll tmp=dsu.find(i);
38        if(root==-1) root=tmp;
39        if(tmp!=root){
40            cout<<"No\n";
41            return;
42        }
43    }
```

```

43     }
44     fp(i,1,n) sort(G[i].begin(),G[i].end());
45     vector<ll> path;
46     auto dfs=[&](auto&&self,ll u) ->void {
47         for(auto &i=cur[u];i<G[u].size();){
48             auto [v,id]=G[u][i];
49             if(vis[id]){
50                 i++;
51                 continue;
52             }
53             vis[id]=1;
54             i++;
55             self(self,v);
56         }
57         path.push_back(u);
58     };
59
60     dfs(dfs,st);
61     for(auto it=path.rbegin();it!=path.rend();it++){
62         cout<<*it<<"\n";
63     }
64 }

```

2.9.2 有向图欧拉路径

```

1 void solve(){
2     ll n,m;
3     cin>>n>>m;
4     vector<vector<pll>> G(n+1);
5     vector<ll> cur(n+1);
6     vector<ll> in(n+1);
7     vector<ll> out(n+1);
8     vector<bool> vis(m+1);
9     ll tot=-1;
10    DSU dsu(n);
11    fp(i,1,m){
12        ll u,v;
13        cin>>u>>v;
14        G[u].push_back({v,++tot});
15        dsu.merge(u,v);
16        in[v]++;
17        out[u]++;
18    }
19    int flag1=0; // out-in=1
20    int flag2=0; // in-out=1
21    int st=1;
22    fp(i,1,n){
23        if(abs(in[i]-out[i])>=2){
24            cout<<"No\n";
25            return;
26        }
27        if(out[i]-in[i]==1) {flag1++;st=i;}
28        if(in[i]-out[i]==1) flag2++;
29    }
30    if(flag1+flag2!=0 and (flag1!=1 or flag2!=1)){

```

```

31     cout<<"No\n";
32     return;
33 }
34 ll root=-1;
35 fp(i,1,n){
36     if(in[i]==0 and out[i]==0) continue;
37     ll tmp=dsu.find(i);
38     if(root==-1) root=tmp;
39     if(tmp!=root){
40         cout<<"No\n";
41         return;
42     }
43 }
44 fp(i,1,n) sort(G[i].begin(),G[i].end());
45 vector<ll> path;
46 auto dfs=[&](auto&&self,ll u) ->void {
47     for(auto &i=cur[u];i<G[u].size();){
48         auto [v,id]=G[u][i];
49         if(vis[id]){
50             i++;
51             continue;
52         }
53         vis[id]=1;
54         i++;
55         self(self,v);
56     }
57     path.push_back(u);
58 };
59
60 dfs(dfs,st);
61 for(auto it=path.rbegin();it!=path.rend();it++){
62     cout<<*it<<" ";
63 }
64 }

```

3. 树

3.1 lca(倍增)

```

1 void solve(){
2     ll n,m,s;
3     cin>>n>>m>>s;
4     vector<vector<ll>> G(n+1);
5     fp(i,1,n-1){
6         ll l,r;
7         cin>>l>>r;
8         G[l].push_back(r);
9         G[r].push_back(l);
10    }
11
12    vector<vector<ll>> fa(n+1,vector<ll>(21,0));
13    vector<ll> dep(n+1);
14
15    auto dfs=[&](auto&& self,ll u,ll d) ->void {

```

```

16     dep[u]=d;
17     for(auto &v:G[u]){
18         if(v==fa[u][0]) continue;
19         fa[v][0]=u;
20         self(self,v,d+1);
21     }
22 };
23
24 dfs(dfs,1,1);
25
26 fp(j,1,20) fp(i,1,n) fa[i][j]=fa[fa[i][j-1]][j-1];
27
28 auto lca=[&](ll a,ll b) ->ll {
29     if(a==b) return a;
30     if(dep[a]<dep[b]) swap(a,b);
31     ll d=dep[a]-dep[b];
32     fp(i,0,20){
33         if(d&(1LL<<i)) a=fa[a][i];
34     }
35     if(a==b) return a;
36     fq(i,20,0){
37         if(fa[a][i]!=fa[b][i]){
38             a=fa[a][i];
39             b=fa[b][i];
40         }
41     }
42     return fa[a][0];
43 };
44
45 fp(i,1,m){
46     ll a,b;
47     cin>>a>>b;
48     cout<<lca(a,b)<<"\n";
49 }
50 }

```

3.2 lca(tarjan)

```

1 void solve(){
2     ll n,m,s;
3     cin>>n>>m>>s;
4     vector<vector<ll>> G(n+1);
5     fp(i,1,n-1){
6         ll x,y;
7         cin>>x>>y;
8         G[x].push_back(y);
9         G[y].push_back(x);
10    }
11    vector<bool> vis(m+1);
12    vector<ll> ans(m+1);
13    vector<ll> fa(n+1);
14    DSU dsu(n+1);
15    vector<vector<array<ll,2>>> qry(m+1);
16    fp(i,1,m){
17        ll a,b;

```

```

18     cin>>a>>b;
19     qry[a].push_back({i,b});
20     qry[b].push_back({i,a});
21 }
22
23 auto dfs=[&](auto&& self, ll u) ->void {
24     for(auto &v:G[u]){
25         if(v==fa[u]) continue;
26         fa[v]=u;
27         self(self,v);
28     }
29     for(auto &[id,x]:qry[u]){
30         if(vis[id]) ans[id]=dsu.find(x);
31         else vis[id]=1;
32     }
33     dsu.merge(u,fa[u]);
34 };
35
36 dfs(dfs,s);
37
38 fp(i,1,m) cout<<ans[i]<<"\n";
39 }

```

3.3 重链剖分

```

1 void solve(){
2     ll n,q,r;
3     cin>>n>>q>>r;
4     // cin>>n>>q; r=1;
5     vector<ll> val(n+1);
6     vector<ll> dep(n+1);
7     vector<ll> fa(n+1);
8     vector<ll> sz(n+1);
9     vector<ll> id(n+1);
10    ll _cnt=0;
11    vector<ll> son(n+1);
12    vector<ll> top(n+1);
13    // LazySegTree seg(n);
14    // fp(i,1,n) cin>>val[i];
15    vector<vector<ll>> G(n+1);
16    fp(i,1,n-1){
17        ll x,y;
18        cin>>x>>y;
19        G[x].push_back(y);
20        G[y].push_back(x);
21    }
22
23    auto dfs1=[&](auto&& self,ll u,ll f,ll d) ->void {
24        dep[u]=d;
25        fa[u]=f;
26        sz[u]=1;
27        ll maxv=0;
28        ll maxid=0;
29        for(auto &v:G[u]){
30            if(v==f) continue;

```

```

31         self(self,v,u,d+1);
32         sz[u]+=sz[v];
33         if(sz[v]>maxv) {
34             maxv=sz[v];
35             maxid=v;
36         }
37     }
38     son[u]=maxid;
39 };
40
41 dfs1(dfs1,r,0,1);
42
43 auto dfs2=[&](auto&& self,ll u,ll t) ->void {
44     top[u]=t;
45     id[u]=++_cnt;
46     if(son[u]>0){
47         self(self,son[u],t);
48     }
49     for(auto &v:G[u]){
50         if(v==fa[u] or v==son[u]) continue;
51         self(self,v,v);
52     }
53 };
54
55 dfs2(dfs2,r,r);
56
57 fp(i,1,n) seg.set(id[i],{val[i]%MOD});
58
59 auto pathadd=[&](ll x,ll y,ll v) {
60     while(top[x]!=top[y]){
61         if(dep[top[x]]<dep[top[y]]) swap(x,y);
62         seg.rangeOperation(id[top[x]],id[x],{v});
63         x=fa[top[x]];
64     }
65     if(dep[x]<dep[y]) swap(x,y);
66     seg.rangeOperation(id[y],id[x],{v});
67 };
68
69 auto pathquery=[&](ll x,ll y){
70     ll ans=0;
71     while(top[x]!=top[y]){
72         if(dep[top[x]]<dep[top[y]]) swap(x,y);
73         ans+=seg.query(id[top[x]],id[x]).val;
74         ans%=MOD;
75         x=fa[top[x]];
76     }
77     if(dep[x]<dep[y]) swap(x,y);
78     ans+=seg.query(id[y],id[x]).val;
79     ans%=MOD;
80     return ans;
81 };
82
83 auto subtreeadd=[&](ll x,ll v){
84     seg.rangeOperation(id[x],id[x]+sz[x]-1,{v});
85 };
86

```

```

87     auto subtreequery=[&](ll x) ->ll {
88         return seg.query(id[x],id[x]+sz[x]-1).val;
89     };
90
91     auto lca=[&](ll x,ll y) ->ll {
92         while(top[x]!=top[y]){
93             if(dep[top[x]]<dep[top[y]]) swap(x,y);
94             x=fa[top[x]];
95         }
96         if(dep[x]<dep[y]) return x;
97         else return y;
98     };
99 }

```

3.4 树上启发式合并

```

1  void solve(){
2      int n;
3      cin>>n;
4      vector<vector<int>> > G(n+1);
5      fp(i,1,n-1){
6          int u,v;
7          cin>>u>>v;
8          G[u].push_back(v);
9          G[v].push_back(u);
10     }
11     vector<ll> c(n+1);
12     vector<ll> fa(n+1);
13     vector<ll> mxson(n+1);
14     vector<ll> sz(n+1);
15     fp(i,1,n) cin>>c[i];
16
17     vector<ll> cnt(n+1);
18     vector<ll> res(n+1);
19     ll ans=0;
20
21     auto dfs1=[&](auto&&self,ll u) ->void {
22         int maxsz=0;
23         sz[u]=1;
24         for(auto &v:G[u]){
25             if(v==fa[u]) continue;
26             fa[v]=u;
27             self(self,v);
28             sz[u]+=sz[v];
29             if(sz[v]>maxsz){
30                 maxsz=sz[v];
31                 mxson[u]=v;
32             }
33         }
34     };
35
36     dfs1(dfs1,1);
37
38     int skip=0;
39

```

```

40     auto update=[&](auto&& self,int u,int val) -> void {
41         if(val==1){
42             if(cnt[c[u]]==0) ans++;
43             cnt[c[u]]++;
44             for(auto &v:G[u]){
45                 if(v!=skip and v!=fa[u]) self(self,v,val);
46             }
47         }
48         else{
49             if(cnt[c[u]]==1) ans--;
50             cnt[c[u]]--;
51             for(auto &v:G[u]){
52                 if(v!=skip and v!=fa[u]) self(self,v,val);
53             }
54         }
55     };
56
57     auto dfs2=[&](auto&& self,ll u,bool keep) ->void {
58         for(auto &v:G[u]){
59             if(v==mxson[u] or v==fa[u]) continue;
60             self(self,v,0);
61         }
62         if(mxson[u]) self(self,mxson[u],1);
63
64         skip=mxson[u];
65         update(update,u,1);
66         res[u]=ans;
67
68         if(!keep) {
69             skip=0;
70             update(update,u,-1);
71         }
72     };
73
74     dfs2(dfs2,1,0);
75
76     int m;
77     cin>>m;
78
79     while(m--){
80         int x;
81         cin>>x;
82         cout<<res[x]<<"\n";
83     }
84 }

```

3.5 树哈希

主要用来判断树是否同构。

```

1  ull mask(ull x) {
2      x ^= x << 13;
3      x ^= x >> 7;
4      x ^= x << 17;
5      return x;

```

```

6 }
7 void solve(){
8     vector<ull> h(n+1); //哈希值
9     auto get_hash=[&](auto&& self,ll u) ->void {
10         h[u] = 1;
11         for (auto v : G[u]) {
12             self(self,v);
13             h[u] += mask(h[v]);
14         }
15     };
16
17     get_hash(get_hash,0);

```

4. 数学

4.1 快速幂和组合数

```

1 vector<ll> fact(N+5);
2 vector<ll> facinv(N+5);
3
4 ll qpow(ll a,ll b,ll m=MOD){
5     ll res=1;
6     while(b){
7         if(b%2) res=res*a%m;
8         b>>=1;
9         a=a*a%m;
10    }
11    return res;
12 }
13
14 void fac(){ //记得调用一下预处理
15     fact[0]=1;
16     for(int i=1;i<=N;++i){
17         fact[i]=fact[i-1]*i%MOD;
18     }
19     facinv[N]=qpow(fact[N],MOD-2);
20     for(int i=N-1;i>=0;--i){
21         facinv[i]=facinv[i+1]*(i+1)%MOD;
22     }
23 }
24
25 ll com(ll a,ll b){
26     if(b>a or b<0) return 0;
27     return fact[a]*facinv[a-b]%MOD*facinv[b]%MOD;
28 }
29
30 ll arr(ll a,ll b){
31     if(b>a or b<0) return 0;
32     return fact[a]*facinv[a-b]%MOD;
33 }

```

4.2 线性筛

```

1 // 只筛素数
2 bitset<N> vis;
3 vector<ll> prime;
4
5 void sieve(){
6     for(int i=2;i<=N-1;++i){
7         if(!vis[i]) prime.push_back(i);
8         for(int j=1;i*prime[j-1]<=N-1;++j){
9             vis[i*prime[j-1]]=1;
10            if(i%prime[j-1]==0) break;
11        }
12    }
13 }
14
15 // 同时找每个数的最小素因子
16 vector<ll> vis(N);
17 vector<ll> prime;
18
19 void sieve(){
20     for(int i=2;i<=N-1;++i){
21         if(!vis[i]) {
22             prime.push_back(i);
23             vis[i]=i;
24         }
25         for(int j=1;i*prime[j-1]<=N-1;++j){
26             vis[i*prime[j-1]]=prime[j-1];
27             if(i%prime[j-1]==0) break;
28         }
29     }
30 }

```

4.3 exgcd

```

1 ll exgcd(ll a,ll b,ll &x,ll &y){
2     if(b==0){
3         x=1;
4         y=0;
5         return a;
6     }
7     ll d=exgcd(b,a%b,y,x);
8     y-=(a/b)*x;
9     return d;
10 }
11
12 void solve(){
13     ll a,b,c,x,y;
14     cin>>a>>b>>c;
15     ll d=exgcd(a,b,x,y);
16     if(c%d){
17         cout<<-1<<"\n";
18         return;
19     }
20     x=x*(c/d);
21     y=y*(c/d);
22     ll flag=0;

```

```

23     ll minx=INF,miny=INF;
24     ll maxx=-INF,maxy=-INF;
25     ll dx=b/d,dy=a/d;
26     ll x1=x,y1=y;
27     if(x1<=0) {
28         ll t=(0-x1)/dx+1;
29         x1+=t*dx;
30         y1-=t*dy;
31     }
32     else{
33         ll t=(x1-1)/dx;
34         x1 -= t*dx;
35         y1 += t*dy;
36     }
37     if(y1>0) flag=1;
38     minx=x1;
39     maxy=y1;
40     ll x2=x,y2=y;
41     if(y2<=0) {
42         ll t=(0-y2)/dy+1;
43         y2+=t*dy;
44         x2-=t*dx;
45     }
46     else{
47         ll t=(y2-1)/dy;
48         y2 -= t*dy;
49         x2 += t*dx;
50     }
51     if(x2>0) flag=1;
52     miny=y2;
53     maxx=x2;
54     if(flag) cout<<(maxx-minx)/dx+1<<" "<<minx<<" "<<miny<<" "<<maxx<<" "
<<maxy<<"\n";
55     else cout<<minx<<" "<<miny<<"\n";
56 }

```

4.4 线性求逆

```

1 vector<ll> inv(N+1);
2
3 void inv_init(){
4     inv[1]=1;
5     for(int i=2;i<=N;++i){
6         inv[i]=(MOD-MOD/i)*inv[MOD%i]%MOD;
7     }
8 }

```

4.5 线性基

```

1 struct XorLinearBasis{
2     int n;
3     vector<ll> base;
4     vector<ll> mask;
5     vector<int> id;

```

```

6
7     xorLinearBasis(int n=63):n(n) {
8         base.assign(n+1,0);
9         mask.assign(n+1,0);
10        id.assign(n+1,0);
11    }
12
13    bool insert(ll val,int idx){
14        ll cur=0;
15        for(int i=n;i>=0;--i){
16            if(!((1LL<<i)&val)) continue;
17            if(!base[i]){
18                base[i]=val;
19                cur^=(1LL<<i);
20                mask[i]=cur;
21                id[i]=idx;
22                return 1;
23            }
24            val ^= base[i];
25            cur ^= mask[i];
26        }
27        return 0;
28    }
29
30    //bool返回是否可行，如果可以，返回要选的数
31    pair<bool,vector<ll>> check(ll x){
32        ll ans=0;
33        for(int i=n;i>=0;--i){
34            if(x&(1LL<<i)){
35                ans^=mask[i];
36                x^=base[i];
37            }
38        }
39        if(x) return {0,{0}};
40        vector<ll> res;
41        for(int i=0;i<=n;++i){
42            if(ans&(1LL<<i)) res.push_back(id[i]);
43        }
44        return {1,res};
45    }
46
47    ll maxsum(){
48        ll ans=0;
49        for(int i=n;i>=0;--i){
50            if(ans&(1LL<<i)) continue;
51            ans ^= base[i];
52        }
53        return ans;
54    }
55
56    };

```

4.6 矩阵类

```
1 struct Mat{
```

```

2     ll n;
3     vector<vector<ll>> d;
4     Mat(ll _n):n(_n) {
5         d.assign(n+1,vector<ll>(n+1,0));
6     }
7     static Mat I(ll n){
8         Mat res(n);
9         for(int i=1;i<=n;++i){
10             res.d[i][i]=1;
11         }
12         return res;
13     }
14     Mat operator+ (const Mat& x) const{
15         Mat res(n);
16         for(int i=1;i<=n;++i){
17             for(int j=1;j<=n;++j){
18                 res.d[i][j]=(d[i][j]+x.d[i][j])%MOD;
19             }
20         }
21         return res;
22     }
23     Mat& operator+= (const Mat& x){
24         for(int i=1;i<=n;++i){
25             for(int j=1;j<=n;++j){
26                 d[i][j]=(d[i][j]+x.d[i][j])%MOD;
27             }
28         }
29         return *this;
30     }
31     Mat operator* (const Mat& x) const{
32         Mat res(n);
33         for(int i=1;i<=n;++i){
34             for(int j=1;j<=n;++j){
35                 for(int k=1;k<=n;++k){
36                     res.d[i][j]+=d[i][k]*x.d[k][j]%MOD;
37                     res.d[i][j]%=MOD;
38                 }
39             }
40         }
41         return res;
42     }
43     static Mat qpow(Mat A,ll b){
44         ll n=A.n;
45         Mat res = I(n);
46         while(b){
47             if(b&1) res=res*A;
48             A=A*A;
49             b>>=1;
50         }
51         return res;
52     }
53     // 返回  $A^m$  和  $S(m-1)=A^0+A^1+A^2+\dots+A^{m-1}$ 
54     static pair<Mat,Mat> powsum(const Mat& A,ll m){
55         ll n=A.n;
56         if(m==0) return {I(n),Mat(n)};
57         if(m==1) return {A,I(n)};

```

```

58         if(m&1){
59             auto [P,S]=powsum(A,m-1);
60             return {P*A,S+P};
61         }
62         else{
63             auto [P,S]=powsum(A,m/2);
64             return {P*P,S*P+S};
65         }
66     }
67 };

```

4.7 FFT

```

1  struct Complex
2  {
3      double x, y;
4      Complex(double _x = 0.0, double _y = 0.0) : x(_x), y(_y) {}
5      Complex operator+(const Complex &b) const { return Complex(x + b.x, y +
b.y); }
6      Complex operator-(const Complex &b) const { return Complex(x - b.x, y -
b.y); }
7      Complex operator*(const Complex &b) const { return Complex(x * b.x - y
* b.y, x * b.y + y * b.x); }
8  };
9
10 class FFT
11 {
12 private:
13     const double PI = acos(-1.0);
14     vector<int> rev;
15     int limit;
16
17     void init(int n)
18     {
19         limit = 1;
20         int l = 0;
21         while (limit <= n)
22         {
23             limit <<= 1;
24             l++;
25         }
26         rev.assign(limit, 0);
27         for (int i = 0; i < limit; i++)
28         {
29             rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
30         }
31     }
32
33     // type = 1 为 FFT, type = -1 为 IFFT
34     void transform(vector<Complex> &A, int type)
35     {
36         for (int i = 0; i < limit; i++)
37         {
38             if (i < rev[i])
39                 swap(A[i], A[rev[i]]);

```

```

40     }
41     for (int mid = 1; mid < limit; mid <= 1)
42     {
43         Complex Wn(cos(PI / mid), type * sin(PI / mid));
44         int R = mid << 1;
45         for (int j = 0; j < limit; j += R)
46         {
47             Complex w(1, 0);
48             for (int k = 0; k < mid; ++k)
49             {
50                 Complex x = A[j + k], y = w * A[j + k + mid];
51                 A[j + k] = x + y;
52                 A[j + k + mid] = x - y;
53                 w = w * Wn;
54             }
55         }
56     }
57     if (type == -1)
58     {
59         for (int i = 0; i < limit; ++i)
60             A[i].x /= limit;
61     }
62 }
63
64 public:
65     vector<ll> multiply(const vector<ll> &a, const vector<ll> &b)
66     {
67         if (a.empty() || b.empty())
68             return {};
69         int n = a.size() - 1, m = b.size() - 1;
70         init(n + m);
71
72         vector<Complex> A(limit), B(limit);
73         for (int i = 0; i <= n; ++i)
74             A[i] = Complex(a[i], 0);
75         for (int i = 0; i <= m; ++i)
76             B[i] = Complex(b[i], 0);
77
78         transform(A, 1);
79         transform(B, 1);
80
81         for (int i = 0; i < limit; i++)
82             A[i] = A[i] * B[i];
83
84         transform(A, -1);
85
86         vector<ll> res(n + m + 1);
87         for (int i = 0; i <= n + m; ++i)
88         {
89             res[i] = (ll)(A[i].x + 0.5);
90         }
91         return res;
92     }
93 };
94
95 void solve()

```

```

96 {
97     FFT fft;
98     ll n, m;
99     cin >> n >> m;
100    vector<ll> f(n + 1);
101    vector<ll> g(m + 1);
102    fp(i, 0, n) cin >> f[i];
103    fp(i, 0, m) cin >> g[i];
104    vector<ll> res = fft.multiply(f, g);
105    fp(i, 0, n + m) cout << res[i] << " ";
106 }

```

4.8 卡特兰数

正方形 $\frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}$

矩形($m \geq n$) $\frac{m-n+1}{m+1} \binom{m+n}{n} = \binom{m+n}{m} - \binom{m+n}{m+1}$

4.9 MillerRabin素性检验

检验一个数是否是素数, 复杂度 $O(7 * \log(n))$ 。

```

1  bool MRtest(ll n){
2      if(n<3 or n%2==0) return n==2;
3      ll u=n-1,t=0;
4      while(u%2==0) u>>=1,t++;
5      vector<ll> base={2,325,9375,28178,450775,9780504,1795265022};
6      for(auto a:base){
7          if(n<=a) break;
8          ll v=qpow(a,u,n);
9          if(v==1 or v==n-1) continue;
10         bool flag=0;
11         for(int j=1;j<t;++j){
12             v=v*v%n;
13             if(v==n-1){
14                 flag=1;
15                 break;
16             }
17         }
18         if(!flag) return 0;
19     }
20     return 1;
21 }
22
23 void solve(){
24     ll n;
25     cin>>n;
26     bool flag=MRtest(n);
27     if(flag) cout<<"Yes\n";
28     else cout<<"No\n";
29 }

```

4.10 CRT

```

1  ll mul(ll a,ll b,ll mod){    //高精度乘法
2      string x=to_string(a);
3      ll res=0;
4      for(int i=0;i<=x.size()-1;++i){
5          res*=10;
6          res+=(x[i]-'0')*b;
7          res%=mod;
8      }
9      return res;
10 }
11
12 void solve(){
13     ll n;
14     cin>>n;
15     vector<ll> a(n+1);    // 模数
16     vector<ll> b(n+1);    // 余数
17     vector<ll> m(n+1);
18     vector<ll> inv(n+1);
19     vector<ll> c(n+1);
20     ll pro=1;
21     for(int i=1;i<=n;++i) {cin>>a[i]>>b[i];pro*=a[i];}
22     for(int i=1;i<=n;++i) {
23         m[i]=(pro/a[i]);
24         ll x,y;
25         exgcd(m[i],a[i],x,y);
26         inv[i]=(x+a[i])%a[i];
27         c[i]=m[i]*inv[i];
28     }
29     ll ans=0;
30     for(int i=1;i<=n;++i){
31         ans=(ans+mul(b[i],c[i],pro)+pro)%pro;
32     }
33     cout<<ans;
34 }

```

5. 字符串

5.1 Hash

```

1  struct custom_hash {
2      static ull splitmix64(ull x) {
3          x += 0x9e3779b97f4a7c15;
4          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5          x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6          return x ^ (x >> 31);
7      }
8      size_t operator()(ull x) const {
9          static const ull FIXED_RANDOM =
10 chrono::steady_clock::now().time_since_epoch().count();
11          return splitmix64(x + FIXED_RANDOM);
12      }
13 };
14 struct Hash {

```

```

15     static inline ll MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
16     static inline ll B1 = 0, B2 = 0;
17     static inline vector<ll> p1 = {1}, p2 = {1};
18
19     static void init(int mx_len) {
20         if (B1 == 0) {
21             mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
22             B1 = uniform_int_distribution<ll>(1e5, 1e9 - 10)(rng) | 1; // 强
制奇数 Base
23             B2 = uniform_int_distribution<ll>(1e5, 1e9 - 10)(rng) | 1;
24         }
25         while (p1.size() <= mx_len) {
26             p1.push_back(p1.back() * B1 % MOD1);
27             p2.push_back(p2.back() * B2 % MOD2);
28         }
29     }
30
31     vector<ll> h1, h2;
32
33     Hash(){}
34
35     //传1-base的字符串
36     Hash(const string& s) {
37         int n = s.size();
38         init(n);
39         h1.resize(n + 1, 0); h2.resize(n + 1, 0);
40         for (int i = 0; i < n; i++) {
41             h1[i + 1] = (h1[i] * B1 + s[i]) % MOD1;
42             h2[i + 1] = (h2[i] * B2 + s[i]) % MOD2;
43         }
44     }
45
46     // 查询子串 [l, r] 的哈希值, 0-indexed 闭区间
47     ull get(int l, int r) const {
48         ll r1 = (h1[r + 1] - h1[l] * p1[r - l + 1]) % MOD1;
49         ll r2 = (h2[r + 1] - h2[l] * p2[r - l + 1]) % MOD2;
50         if (r1 < 0) r1 += MOD1;
51         if (r2 < 0) r2 += MOD2;
52         return ((ull)r1 << 32) | r2; // 高32位放hash1, 低32位放hash2
53     }
54 };
55
56 void solve(){
57     ll n;
58     cin>>n;
59     vector<string> s(n+1);
60     fp(i,1,n) cin>>s[i];
61     fp(i,1,n) s[i]=" "+s[i];
62     vector<Hash> hash(n+1);
63     fp(i,1,n) hash[i]=Hash(s[i]);
64     vector<ll> l(n+1);
65     fp(i,1,n) l[i]=(int)s[i].size()-1;
66     ll ans=0;
67     gp_hash_table<ull, ll, custom_hash> cnt;
68     fp(i,1,n){

```

```

69         fq(j,l[i],1){
70             ans+=cnt[hash[i].get(j,l[i])];
71         }
72         fp(j,1,l[i]){
73             cnt[hash[i].get(1,j)]++;
74         }
75     }
76 }

```

5.2 KMP

```

1  vector<ll> ans;
2
3  void kmp(string a,string b){
4      string s=" "+b+"#"+a;
5      vector<ll> pi(s.size(),0);
6      for(int i=2;i<=s.size()-1;++i){
7          ll len=pi[i-1];
8          while(len and s[i]!=s[len+1]){
9              len=pi[len];
10         }
11         if(s[i]==s[len+1]){
12             pi[i]=len+1;
13         }
14         if(pi[i]==b.size()){
15             ans.push_back(i-2*b.size());
16         }
17     }
18 }

```

5.3 Manacher

```

1  void solve(){
2      string s1;
3      cin>>s1;
4      int len=s1.size();
5      vector<int> p(2*len+2);
6      vector<char> s(2*len+2);
7      s[0]=' ';
8      s[1]='*';
9      for(int i=0;i<=len-1;++i){
10         s[2+2*i]=s1[i];
11         s[3+2*i]='*';
12     }
13     int maxpos=0;
14     int mid=0;
15     for(int i=1;i<=2*len+1;++i){
16         if(maxpos>=i){
17             ll k=2*mid-i;
18             p[i]=min(p[k],maxpos-i+1);
19         }
20         else{
21             p[i]=1;
22         }

```

```

23     while(i-p[i]>=1 and i+p[i]<=len*2+1 and s[i-p[i]]==s[i+p[i]]){
24         p[i]++;
25     }
26     if(i+p[i]-1>maxpos){
27         maxpos=i+p[i]-1;
28         mid=i;
29     }
30 }
31 int ans=0;
32 for(int i=1;i<=2*len+1;++i){
33     ans=max(ans,(2*p[i]-1)/2);
34 }
35 cout<<ans;
36 }

```

6.4 Trie

```

1  struct Trie{
2      static constexpr int SIZE = 62;
3      struct Node{
4          int ch[SIZE] = {0};
5          int pass=0;
6          int end=0;
7          Node() = default;
8      };
9      vector<Node> nodes;
10     Trie(){
11         nodes.reserve(200000);
12         nodes.emplace_back();
13     }
14
15     int index(const char& ch) const {
16         if(ch >='0' and ch<='9'){
17             return ch-'0';
18         }
19         if(ch>='a' and ch<='z'){
20             return ch-'a'+10;
21         }
22         if(ch>='A' and ch<='Z'){
23             return ch-'A'+36;
24         }
25     }
26
27     void insert(const string& s){
28         int cur=0;
29         for(char ch:s){
30             int idx = index(ch);
31             if(nodes[cur].ch[idx]==0){
32                 nodes[cur].ch[idx]=nodes.size();
33                 nodes.emplace_back();
34             }
35             cur=nodes[cur].ch[idx];
36             nodes[cur].pass++;
37         }
38         nodes[cur].end++;

```

```

39     }
40
41     // 精确查找一个单词是否存在
42     bool find(const string& s){
43         int cur=0;
44         for(char ch:s){
45             int idx = index(ch);
46             if(nodes[cur].ch[idx]==0){
47                 return 0;
48             }
49             cur=nodes[cur].ch[idx];
50         }
51         return nodes[cur].end > 0;
52     }
53
54     // 查找是否存在以指定prefix为前缀的单词
55     int start(const string& s){
56         int cur=0;
57         for(char ch:s){
58             int idx = index(ch);
59             if(nodes[cur].ch[idx]==0){
60                 return 0;
61             }
62             cur=nodes[cur].ch[idx];
63         }
64         return nodes[cur].pass;
65     }
66 };
67
68 void solve(){
69     ll n,q;
70     cin>>n>>q;
71     Trie trie;
72     while(n--){
73         string s;
74         cin>>s;
75         trie.insert(s);
76     }
77     while(q--){
78         string s;
79         cin>>s;
80         cout<<trie.start(s)<<"\n";
81     }
82 }

```

6.5 ACAutomation

```

1  struct ACAutomation{
2      static constexpr int SIZE = 26;
3      struct Node{
4          int ch[SIZE] = {0};
5          int fail=0;
6          int count=0; //文本串经过该节点的次数
7          int end=0;
8          int match=0;

```

```

9      vector<int> has;
10      Node() = default;
11  };
12  vector<Node> nodes;
13  vector<int> topo;
14  int tot=0;
15
16  ACAutomation(){
17      nodes.reserve(200000);
18      nodes.emplace_back();
19  }
20
21  int index(const char& ch) const {
22      if(ch>='a' and ch<='z'){
23          return ch-'a';
24      }
25  }
26
27  void insert(const string& s){
28      int cur=0;
29      for(char ch:s){
30          int idx = index(ch);
31          if(nodes[cur].ch[idx]==0){
32              nodes[cur].ch[idx]=nodes.size();
33              nodes.emplace_back();
34          }
35          cur=nodes[cur].ch[idx];
36      }
37      nodes[cur].end++;
38      nodes[cur].has.push_back(++tot);
39  }
40
41  void build(){
42      queue<int> q;
43      for(int i=0;i<SIZE;++i){
44          if(nodes[0].ch[i]){
45              q.push(nodes[0].ch[i]);
46          }
47      }
48      while(q.size()){
49          int u=q.front();
50          topo.push_back(u);
51          q.pop();
52          nodes[u].match=nodes[u].end+nodes[nodes[u].fail].match;
53          for(int i=0;i<SIZE;++i){
54              if(nodes[u].ch[i]){
55                  nodes[nodes[u].ch[i]].fail=nodes[nodes[u].fail].ch[i];
56                  q.push(nodes[u].ch[i]);
57              }
58              else{
59                  nodes[u].ch[i]=nodes[nodes[u].fail].ch[i];
60              }
61          }
62      }
63  }
64

```

```

65     void query(const string& s){
66         int cur=0;
67         for(char ch:s){
68             int idx=index(ch);
69             cur=nodes[cur].ch[idx];
70             nodes[cur].count++;
71         }
72         vector<ll> ans(tot+1);
73         for(auto it=topo.rbegin();it!=topo.rend();it++){
74             int i=(*it);
75             for(auto x:nodes[i].has) ans[x]=nodes[i].count;
76             nodes[nodes[i].fail].count+=nodes[i].count;
77         }
78         for(int i=1;i<=tot;++i) cout<<ans[i]<<"\n";
79     }
80 };
81
82 void solve(){
83     ll n;
84     cin>>n;
85     ACAutomation ac;
86     fp(i,1,n){
87         string s;
88         cin>>s;
89         ac.insert(s);
90     }
91     ac.build();
92     string s;
93     cin>>s;
94     ac.query(s);
95 }

```

6.6 SA

```

1  struct SA{
2      string s;
3      int n;
4      vector<int> sa;
5      vector<int> rk;
6      vector<int> height;
7
8      //传0-base的字符串
9      SA(const string& str){
10         n=str.size();
11         s=" "+str;
12         sa.resize(n+1);
13         rk.resize(2*n+1);
14         height.resize(n+1);
15         build();
16     }
17
18     void build(){
19         int m=255; //计数桶的个数
20         vector<int> oldrk(2*n+1,0);
21         vector<int> cnt(max(n,m)+1,0);

```

```

22     vector<int> id(n+1);
23     for(int i=1;i<=n;++i){
24         rk[i]=s[i];
25         cnt[rk[i]]++;
26     }
27     for(int i=1;i<=m;++i){
28         cnt[i]+=cnt[i-1];
29     }
30     for(int i=n;i>=1;--i){
31         sa[cnt[rk[i]]--]=i;
32     }
33     for(int k=1;k<n;k*=2){
34         int num=0;
35         for(int i=n-k+1;i<=n;++i) id[++num]=i;
36         for(int i=1;i<=n;++i){
37             if(sa[i]>k) id[++num]=sa[i]-k;
38         }
39
40         fill(cnt.begin(),cnt.begin()+m+1,0);
41         for(int i=1;i<=n;++i){
42             cnt[rk[id[i]]]++;
43         }
44         for(int i=1;i<=m;++i){
45             cnt[i]+=cnt[i-1];
46         }
47         for(int i=n;i>=1;--i){
48             sa[cnt[rk[id[i]]]--]=id[i];
49         }
50
51         swap(rk,olldrk);
52         int p=0;
53         for(int i=1;i<=n;++i){
54             if(olldrk[sa[i]]==olldrk[sa[i-1]] and olldrk[sa[i] + k] ==
olldrk[sa[i - 1] + k]){
55                 rk[sa[i]] = p;
56             }
57             else{
58                 rk[sa[i]] = ++p;
59             }
60         }
61
62         if(p==n) break;
63         m=p;
64     }
65
66     if(n==1) rk[1]=1;
67
68     int k=0;
69     for(int i=1;i<=n;++i){
70         if(rk[i]==1) {
71             k=0;
72             height[rk[i]]=0;
73             continue;
74         }
75         if(k) k--;
76         int j=sa[rk[i]-1];

```

```

77         while(i+k<=n and j+k<=n and s[i+k]==s[j+k]) k++;
78         height[rk[i]]=k;
79     }
80 }
81 };
82
83 void solve(){
84     string s;
85     cin>>s;
86     SA sa(s);
87     int n=s.size();
88     fp(i,1,n) cout<<sa.sa[i]<<" ";
89 }

```

6. DP

6.1 子集枚举

单纯子集枚举为 $O(3^n)$ 的复杂度。

```

1 void solve(){
2     ll n,c;
3     cin>>n>>c;
4     vector<ll> w(n);
5     fp(i,0,n-1) cin>>w[i];
6     vector<bool> valid(1LL<<n);
7     vector<ll> dp(1LL<<n,INF);
8     dp[0]=0;
9     fp(i,1,(1LL<<n)-1){
10         ll cur=0;
11         fp(j,0,n-1) if((1LL<<j)&i) cur+=w[j];
12         if(cur<=c) valid[i]=1;
13     }
14     fp(mask,1,(1LL<<n)-1){
15         for(int sub=mask;sub;sub = ((sub-1) & mask)){
16             if(valid[sub]){
17                 dp[mask]=min(dp[mask],dp[mask^sub]+1);
18             }
19         }
20     }
21     cout<<dp[(1LL<<n)-1];
22 }

```

6.2 斜率优化

```

1 void solve(){
2     ll n,c;
3     cin>>n>>c;
4     vector<ll> h(n+1);
5     fp(i,1,n) cin>>h[i];
6     vector<ll> dp(n+1);

```

```

7     vector<ll> x(n+1);
8     vector<ll> y(n+1);
9     // xi=hi yi=hi^2+dp[i]
10
11     deque<p11> pnt; // point
12     auto check = [&](const p11 & u, const p11 & v, const p11 & w){ //如果要
删, return 1
13         p11 k1 = {v.fi - u.fi, v.se - u.se};
14         p11 k2 = {w.fi - v.fi, w.se - v.se};
15         //根据题意调整是否可以取等, 不取等会保留相同斜率
16         return ((ll)k1.se * k2.fi > (ll)k1.fi * k2.se);
17     };
18
19     dp[1]=0;
20     x[1]=h[1];
21     y[1]=h[1]*h[1]+dp[1];
22     pnt.push_back({x[1],y[1]});
23     fp(i,2,n){
24         while(pnt.size()>=2){
25             const p11 & u = pnt.front();
26             const p11 & v = pnt[1];
27             ll k = 2 * h[i];
28             p11 k1 = {v.fi - u.fi, v.se - u.se};
29             if(k1.se < (ll) k1.fi * k) pnt.pop_front();
30             else break;
31         }
32         dp[i]=h[i]*h[i]+(pnt[0].se-2*h[i]*pnt[0].fi)+c;
33         x[i]=h[i];
34         y[i]=h[i]*h[i]+dp[i];
35         while(pnt.size()>=2 and check(pnt[pnt.size()-2],pnt.back(),
{x[i],y[i]})){
36             pnt.pop_back();
37         }
38         pnt.push_back({x[i],y[i]});
39     }
40     cout<<dp[n]<<"\n";
41 }

```

7. 博弈论

7.1 Nim博弈

给定 n 堆物品, 第 i 堆物品有 a_i 个。两名玩家分别行动, 每次可以任选一堆, 取出任意多个物品, 可以一把取光但是不能不取。取走最后一个物品的人胜利。

先手必胜, 当且仅当 $a_1 \oplus a_2 \oplus \dots \oplus a_n = 0$ 。

7.2 Wythoff博弈

有两堆石子, 石子数可以不同。两人轮流取石子, 每次可以在一堆中取, 或者从两堆中取走相同个数的石子, 数量不限, 取走最后一个石头的人获胜。判定先手是否必胜。

假设两堆石子为 (x, y) , $x < y$ 。

那么先手必败, 当且仅当:

$$(y - x) * \frac{\sqrt{5}+1}{2} = x$$

下面是使用整数，避免浮点误差的做法。

```

1 void solve(){
2     ll x,y;
3     cin>>x>>y;
4     if(x>y) swap(x,y);
5     ll k=y-x;
6     if((2*x-k)*(2*x-k)<=5*k*k and 5*k*k<(2*x+2-k)*(2*x+2-k)){
7         cout<<0<<"\n"; //先手必败
8     }
9     else{
10         cout<<1<<"\n"; //先手必胜
11     }
12 }

```

8. 杂项

8.1 最长上升子序列

```

1 void solve(){
2     ll n;
3     cin>>n;
4     vector<ll> a(n+1);
5     for(int i=1;i<=n;++i) {cin>>a[i];}
6     vector<ll> q;
7     for(int i=1;i<=n;++i){
8         // 严格上升用lower_bound, 单调不减用upper_bound
9         auto it=lower_bound(q.begin(),q.end(),a[i]);
10        if(it==q.end()) q.push_back(a[i]);
11        else {
12            *it=a[i];
13        }
14    }
15    cout<<q.size();
16 }

```

8.2 三分

8.2.1 三分 (三等分)

```

1 vector<double> coe(N);
2 ll n;
3
4 double f(double x){
5     double res=0;
6     for(int i=n;i>=0;--i){
7         res+=coe[i]*pow(x,i);
8     }
9     return res;
10 }
11

```

```

12 void solve(){
13     cin>>n;
14     double l,r;
15     cin>>l>>r;
16     double l1,r1;
17     l1=l+(r-l)/3;
18     r1=r-(r-l)/3;
19     for(int i=n;i>=0;--i){
20         cin>>coe[i];
21     }
22     while(r1-l1>eps){
23         l1=l+(r-l)/3;
24         r1=r-(r-l)/3;
25         double f11=f(l1);
26         double fr1=f(r1);
27         if(f11>=fr1){
28             r=r1;
29         }
30         else{
31             l=l1;
32         }
33     }
34     cout<<l1;
35 }

```

8.2.2 三分 (近似三分)

```

1  vector<double> coe(N);
2  ll n;
3
4  double f(double x){
5      double res=0;
6      for(int i=n;i>=0;--i){
7          res+=coe[i]*pow(x,i);
8      }
9      return res;
10 }
11
12 void solve(){
13     cin>>n;
14     double l,r;
15     cin>>l>>r;
16     double l1,r1;
17     l1=(l+r)/2-eps;
18     r1=(l+r)/2+eps;
19     for(int i=n;i>=0;--i){
20         cin>>coe[i];
21     }
22     ll cnt=0;
23     while(cnt<=100){
24         cnt++;
25         l1=(l+r)/2-eps;
26         r1=(l+r)/2+eps;
27         double f11=f(l1);
28         double fr1=f(r1);

```

```

29         if(f11>=fr1){
30             r=r1;
31         }
32         else{
33             l=l1;
34         }
35     }
36     cout<<l1;
37 }

```

8.2.3 三分 (整数三分)

```

1 void solve(){
2     ll n,m;
3     cin>>n>>m;
4     vector<ll> a(n+1);
5     vector<ll> b(m+1);
6     for(int i=1;i<=n;++i){
7         cin>>a[i];
8     }
9     for(int i=1;i<=m;++i){
10        cin>>b[i];
11    }
12    sort(a.begin()+1,a.end());
13    sort(b.begin()+1,b.end());
14    vector<ll> asum(n+1);
15    vector<ll> bsum(m+1);
16    for(int i=1;i<=n;++i){
17        asum[i]=asum[i-1]+a[n+1-i]-a[i];
18    }
19    for(int i=1;i<=m;++i){
20        bsum[i]=bsum[i-1]+b[m+1-i]-b[i];
21    }
22    vector<ll> ans;
23    for(ll k=1;max(0LL,2*k-n)<=min(k,m-k);++k){
24        ll l=max(0LL,2*k-n);
25        ll r=min(k,m-k);
26        auto f=[&](ll u){return bsum[u]+asum[k-u];};
27        while(r-l>3){
28            ll m1=(2*l+r)/3;
29            ll mr=(l+2*r)/3;
30            if(f(m1)<f(mr)){
31                l=m1;
32            }
33            else{
34                r=mr;
35            }
36        }
37        ll res=0;
38        for(int i=l;i<=r;++i){
39            res=max(res,f(i));
40        }
41        ans.push_back(res);
42    }
43    cout<<ans.size()<<"\n";

```

```

44     for(auto &p:ans) cout<<p<<" ";
45     cout<<"\n";
46 }

```

8.3 CDQ分治

```

1  struct it{
2      ll a,b,c,cnt,ans;
3  };
4
5  void solve(){
6      ll n,k;
7      cin>>n>>k;
8      vector<array<ll,3>> a(n+1);
9      fp(i,1,n){
10         cin>>a[i][0]>>a[i][1]>>a[i][2];
11     }
12     sort(a.begin()+1,a.end());
13     ll m=0;
14     vector<it> its;
15     its.push_back({0,0,0,0,0});
16     fp(i,1,n){
17         if(a[i][0]==a[i-1][0] and a[i][1]==a[i-1][1] and a[i][2]==a[i-1][2])
18         {
19             its[m].cnt++;
20         }
21         else{
22             m++;
23             its.push_back({a[i][0],a[i][1],a[i][2],1,0});
24         }
25     }
26     vector<it> tmp(m+1);
27     Fenwick<ll> bit(k);
28     auto solve=[&](auto&& self,ll l,ll r) ->void {
29         if(l==r) return;
30         ll mid=l+r>>1;
31         self(self,l,mid);
32         self(self,mid+1,r);
33         ll pnt1=l;
34         ll pnt2=mid+1;
35         ll pnt3=l;
36         vector<pll> op;
37         while(pnt1<=mid and pnt2<=r){
38             if(its[pnt1].b<=its[pnt2].b){
39                 bit.add(its[pnt1].c,its[pnt1].cnt);
40                 op.push_back({its[pnt1].c,its[pnt1].cnt});
41                 tmp[pnt3]=its[pnt1];
42                 pnt1++;
43                 pnt3++;
44             }
45             else{
46                 its[pnt2].ans+=bit.rangeSum(1,its[pnt2].c);
47                 tmp[pnt3]=its[pnt2];
48                 pnt2++;
49                 pnt3++;

```

```

49         }
50     }
51     while(pnt1<=mid){
52         tmp[pnt3]=its[pnt1];
53         pnt1++;
54         pnt3++;
55     }
56     while(pnt2<=r){
57         its[pnt2].ans+=bit.rangeSum(1,its[pnt2].c);
58         tmp[pnt3]=its[pnt2];
59         pnt2++;
60         pnt3++;
61     }
62     for(auto [pos,cnt]:op) bit.add(pos,-cnt);
63     fp(i,l,r) its[i]=tmp[i];
64 };
65 solve(solve,1,m);
66 fp(i,1,m){
67     its[i].ans+=its[i].cnt-1;
68 }
69 vector<ll> ans(n);
70 fp(i,1,m) ans[its[i].ans]+=its[i].cnt;
71 fp(i,0,n-1) cout<<ans[i]<<"\n";
72 }

```

8.4 离散化

```

1 sort(vec.begin(), vec.end());
2 vec.erase(unique(vec.begin(), vec.end()), vec.end());

```

8.5 随机数

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 ll rand(ll l, ll r) {
4     uniform_int_distribution<ll> distribution(l, r);
5     return distribution(rng);
6 }

```

8.6 川的输出

```

1 void solve(){
2     ll x;
3     cin>>x;
4     ll x1=(ll) x*x;
5     cout << format("{} ", x1) << "\n";
6 }

```

8.7 对拍

```
1  int main(){
2      ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
3      ll cnt=0;
4      while(1){
5          system("generator.exe > in.txt");
6          system("std.exe < in.txt > std.txt");
7          system("my.exe < in.txt > my.txt");
8          if(system("fc std.txt my.txt")) break;
9          cout<<"AC"<<cnt<<endl;
10     }
11     cout<<"WA"<<endl;
12     return 0;
13 }
```

bat版

```
1  @echo off
2  echo Compiling programs...
3  g++ generator.cpp -o generator.exe -O2 -std=c++23
4  g++ std.cpp -o std.exe -O2 -std=c++23
5  g++ my.cpp -o my.exe -O2 -std=c++23
6
7  if errorlevel 1 (
8      echo Compilation failed!
9      pause
10     exit
11 )
12
13 echo Compilation successful!
14
15 :: 对拍循环
16 set /a count=0
17 :loop
18 set /a count+=1
19 echo Testing case %count%...
20
21 :: 生成数据
22 generator.exe > data.in
23
24 :: 运行两个程序
25 std.exe < data.in > std.out
26 my.exe < data.in > my.out
27
28 :: 比较输出
29 fc std.out my.out > nul
30
31 if errorlevel 1 (
32     echo =====
33     echo Found different output at case %count%!
34     echo =====
35     echo Input data saved in: data.in
36     echo Standard output saved in: std.out
```

```
37     echo Your output saved in: my.out
38     pause
39     exit
40 ) else (
41     echo Case %count% passed.
42 )
43
44 goto loop
```

8.8 设置输出精度

```
1 cout << fixed << setprecision(15);
```

8.9 pbds的平衡树

```
1 tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
  tr;
```